

MetaSignal

AIPM Case Study — Experiment Validity Platform

Product Design Narrative · Sidharth Kriplani · 2025

What this document is

This is a product design narrative — five sections that walk through why MetaSignal exists, what was deliberately built and what was deliberately left out, what problem the system is actually solving, how you'd know whether it's working, and where it honestly falls short. It's written for another PM, not an engineer.

MetaSignal is a production-simulated experimentation intelligence platform. It is not a live system with real users. It is an offline audit tool — the kind of infrastructure a platform team would build to give data scientists and PMs confidence that an experiment result is trustworthy before anyone makes a ship or hold decision. Every design choice described here is a product decision first, a technical one second.

Contents

- [01 The Problem](#)
- [02 The Product Decisions](#)
- [03 The Failure Mode Addressed](#)
- [04 What Success Looks Like](#)
- [05 One Honest Gap](#)

Who was making the wrong decision, and what were they getting wrong

The users this system is designed for are data scientists and product managers at tech companies who run A/B experiments. They are not statistics amateurs — they know what a p-value is, they know their experiment framework, and they have shipped products based on experiment results. The problem is not that they lack knowledge. The problem is that the systems they depend on allow them to make structurally invalid decisions without surfacing any warning.

Three compounding failure modes

Peeking before sample size is reached

Most experiment frameworks expose live p-values as the experiment runs. There is no enforcement of a minimum runtime or a sample size requirement. A PM whose feature is showing a positive trend on day three has every incentive to call the experiment early. The result is inflated Type I error — effects that look real, are declared significant, and then fail to replicate. Real improvements get killed because the next experiment "didn't see the same effect," when the first result was simply noise.

Sample ratio mismatches that nobody checked

A sample ratio mismatch (SRM) occurs when the treatment and control groups contain significantly different proportions of users than the assignment mechanism intended. The causes are mundane: a browser extension filtering users in one arm, a cache misconfiguration affecting mobile users, an SDK version gate that split the assignment bucket. The symptom is that the "experiment" is not measuring what you think — one arm is systematically missing a population segment. Downstream metric differences are real, but they reflect the assignment bias, not the treatment effect. Teams that don't check for SRM before reading results are making decisions on invalid data.

Variance that CUPED could have reduced by 40–60%

CUPED (Controlled-experiment Using Pre-Experiment Data) is a variance reduction technique that uses pre-experiment covariate data to reduce noise in the outcome metric. A well-implemented CUPED reduces variance by 40–60% in typical e-commerce and consumer product settings — meaning the same sample size now has meaningfully more statistical power. Teams that skip CUPED are running underpowered experiments. An underpowered experiment misses real effects: a genuine 5% revenue improvement fails to clear significance because the confidence interval is too wide. That feature doesn't ship. The product gets worse by not improving.

The consequence: real product improvements were killed because the signal was too noisy, and regressions shipped because the experiment wasn't long enough to detect them. Neither outcome was intentional — both were preventable.

These three problems are not exotic edge cases. They are the default behavior of most experiment setups that were built for speed rather than rigor. The gap MetaSignal fills is: check validity before anyone reads the result.

What was built, what was deliberately left out, and why each boundary is a product decision — not a technical one

What was built

SRM Detection

A chi-square goodness-of-fit test that compares actual assignment proportions against intended proportions before any metric is surfaced. If the test fires, the experiment is invalidated before anyone sees the readout. The product decision here: SRM is a gate, not a warning. An experiment that fails SRM produces no metric output.

Peeking Guard

Statistical flags for experiments where significance status has flipped across interim reads — the signature of p-value fishing. This is surfaced as a Boolean on the experiment evaluation artifact so it is part of the permanent record, not a transient dashboard state.

CUPED Variance Reduction

Pre-experiment covariate regression applied before the treatment effect is computed. The product value is not the math — it is that analysts can no longer accidentally run un-adjusted analyses without it being visible. CUPED output is the canonical result; unadjusted numbers are not surfaced.

Guardrail-First Metric Ordering

Guardrail metrics (latency, error rate, revenue, delayed conversion) are evaluated before the primary metric. A positive primary metric is blocked by a guardrail violation. The product decision: a marginal engagement lift is never sufficient reason to ship a latency regression. The system makes this structurally impossible to override without a written, logged justification.

Right-Censored Revenue

Delayed guardrail metrics — return rates, 30-day LTV, long-attribution conversion — are marked as immature until their attribution window closes. A naive implementation treats an incomplete 30-day window as zero, which looks like a revenue drop. This prevents the guardrail from firing on incompleteness rather than actual regression.

Streaming Anomaly Detection

A provisional early-warning layer that monitors for SRM drift, consumer lag spikes, late events, and duplicate detection in near-real-time. These signals are provisional — they trigger investigation, never a final decision. Batch remains authoritative. The product value: anomalies surface during the experiment, not after the readout.

Metric Registry

A versioned, conflict-detectable definition store for every metric. Definitions store numerator SQL, denominator SQL, denominator type, grain, entity type, and a config hash. If two active metrics in the same family use different denominators, the system flags a conflict. The product value: metric definitions are governed artifacts, not analyst conventions.

What was deliberately not built

Automated ship decisions

MetaSignal flags. Humans decide. This was not a resource constraint — it was a deliberate product boundary. Automating the ship decision would require the system to hold opinions about roadmap priority, competitive timing, and acceptable risk that it has no basis to hold. The system's job is to tell you whether the result is trustworthy. What you do with a trustworthy result is a business decision.

Bayesian inference

The system uses frequentist statistics throughout. This was deliberate. Frequentist output — p-values, confidence intervals, significance flags — is legible to a non-statistician PM who needs to present results in a product review. Bayesian posteriors require interpretation that most stakeholder audiences are not equipped for. The product constraint was auditability by humans who are not Bayesian practitioners, not a technical preference.

Real traffic integration

MetaSignal is an offline audit tool. It does not sit in the assignment path, does not capture events in real time, and does not have write access to experiment infrastructure. This is a scope boundary, not a gap. A platform that tries to own both the audit layer and the assignment layer is taking on operational risk and organizational complexity that an audit tool should not carry. The product's value proposition is: bring your experiment data, get a validity verdict.

Validity before significance

The failure mode MetaSignal was designed to address is specific: teams checking p-values before checking whether the experiment was run correctly.

The standard experiment readout flow at most companies goes: open the dashboard, read the primary metric, check significance, decide. The validity questions — was the assignment clean? was there an SRM? was the experiment long enough? is the guardrail metric mature? — get asked only when someone is skeptical of the result, not as a precondition to reading it.

This creates a predictable failure pattern. A positive result gets scrutinized enough to ship. A negative result gets scrutinized enough to restart. Neither gets scrutinized for whether the randomization was broken, whether a delayed metric is still accumulating, or whether significance was reached by peeking on day three of a planned fourteen-day experiment.

MetaSignal answers one question: "Is this experiment result trustworthy enough to act on?" It does not answer "should you ship?" Those are different questions, and conflating them is how validity problems get ignored.

The system enforces validity-before-significance structurally. The pipeline runs SRM check before CUPED evaluation. CUPED evaluation runs before guardrail assessment. Guardrail assessment runs before a decision recommendation is generated. No step in this chain can be skipped. A team cannot accidentally read a p-value on an experiment that has a sample ratio mismatch — the system will not produce a p-value for that experiment.

What the system detects

- Sample ratio mismatch via chi-square goodness-of-fit (invalidates the experiment)
- Peeking — significance that flipped across interim reads (flagged on the evaluation record)
- Novelty effect — week-one lift that is substantially larger than week-two lift (flagged)
- Simpson's paradox — aggregate positive effect masking segment-level regressions (flagged)
- Guardrail breach — any guardrail in violation blocks ship regardless of primary effect

- Right-censored guardrail immaturity — delayed metric not yet fully attributed (held, not failed)
- Streaming anomalies — consumer lag, duplicate events, late events, SRM drift in near-real-time
- Metric definition conflicts — same metric family, different denominators across active versions

Product metrics, not model metrics

A common mistake in platforms built by engineers is measuring what the system does rather than measuring whether the system is solving the problem. MetaSignal is not successful because its SRM chi-square test has a 5.5% false-positive rate on A/A runs. It is successful if teams are making fewer wrong ship decisions.

The following are the product metrics that would tell you whether MetaSignal is working — each framed as a measurable outcome, not a system property.

Reduction in false ship decisions

The most important metric, and the hardest to measure directly. A proxy: track the rate at which shipped features are rolled back within 30 days due to metric regression that was present in the experiment data. If MetaSignal is catching guardrail breaches and SRM before readout, this rate should decline. Baseline measurement requires historical data on rollbacks and their root causes.

Reduction in invalid experiment restarts

An experiment restart is costly: new assignment, new sample accumulation, new readout cycle. Restarts driven by post-hoc discovery of SRM or peeking are entirely preventable. Track the fraction of experiment restarts attributable to validity problems that MetaSignal's checks would have surfaced before the readout. Reduction in this rate is a direct product success signal.

Time saved per experiment readout cycle

Current state at most teams: a readout meeting involves manual validity checks, ad-hoc statistical review, and disagreements about whether to trust the result. MetaSignal produces a validity verdict before the meeting. Measure time from experiment close to go/no-go decision. Target: readout cycle time is dominated by discussion of the result, not interrogation of whether the result is valid.

Retroactive SRM audit coverage

One immediate use case that does not require real-time integration: run past experiments through the SRM detector to identify which historical ship decisions were made on experiments with sample ratio mismatches. This is an audit function. Success metric: fraction of the historical experiment corpus that has been audited, and number of past decisions that should be revisited. This creates organizational trust in the audit tool before it is used prospectively.

Peeking rate reduction

If experiments are being called early, the distribution of experiment durations will show a spike at short runtimes and a depletion at the planned duration. Track the fraction of experiments called before their planned sample size was reached, before and after MetaSignal's peeking guard is in place. Reduction in short-runtime calls is a signal that the peeking guard is changing behavior.

These metrics share a property: they measure outcomes for the people making experiment decisions, not outputs from the system producing the validity checks. A system that generates lots of flags no one looks at is not successful.

What this system cannot do, and why that was the right call

MetaSignal does not make ship recommendations.

It audits validity. A team could clear every MetaSignal check — no SRM, no peeking, CUPED applied, all guardrails green, anomaly detection quiet — and still make the wrong ship decision based on business context that MetaSignal cannot see.

The business context that a ship decision requires but that MetaSignal cannot hold:

- Roadmap priority — is this experiment's feature in the critical path for the next quarter?
- Competitive timing — does a three-week delay to collect more sample cost market position?
- Resource cost — does shipping require infrastructure investment that changes the ROI threshold?
- Risk tolerance — is the team in a growth phase where a 0.5% revenue risk is acceptable, or in a consolidation phase where it is not?
- Organizational trust — has the PM team built enough statistical credibility to act on a marginal result, or does it need a stronger signal to get buy-in?

Automating the ship decision would require encoding business context that an experiment audit tool should not own. The moment MetaSignal recommends shipping, it has implicitly made assumptions about roadmap priority, risk appetite, and competitive timing that belong to the product team — not to the audit layer.

This was a deliberate scope boundary, not a gap in ambition. The right abstraction for an experiment audit tool is: here is everything I can tell you about whether your result is trustworthy; the decision about what to do with a trustworthy result is yours. Any tool that collapses the validity question and the business decision into a single automated output is hiding the business judgment behind the statistical output — which is exactly the failure mode MetaSignal was built to surface, not to reproduce.

The practical implication: MetaSignal works best when the team using it has a clear decision framework for what to do once validity is confirmed. If a team does not know what a "good enough" experiment result looks like for their specific product context, MetaSignal's validity audit will produce a valid result that still generates disagreement. That disagreement belongs in a different conversation — one about

business strategy, not experiment methodology.

MetaSignal is one layer of a complete experiment culture. It owns the validity layer. Everything above that — what you ship, when you ship it, what tradeoffs you accept — belongs to the people who understand the product, the business, and the users.